Spark Performance Optimization Analysis

DSCC-402 Final Project
Tasin Tayeba Khan

This document analyzes Spark performance characteristics of a four-layer tweet sentiment pipeline (Bronze → Silver → Gold → Application) deployed on Databricks Serverless. The pipeline ingests approximately 50,000 raw tweets from S3, applies text preprocessing and mention extraction, runs distributed ML inference using a distilbert sentiment model, and materializes aggregated analytics for dashboarding. Performance analysis focuses on four key areas: UDF serialization overhead, shuffle behavior in aggregations, data skew in groupBy operations, and Delta table storage optimization.

1. Skew

1.1 Observation

The silver layer uses explode_outer() to expand mention arrays, and the gold aggregation groups by mention. Data skew can occur when certain mentions appear disproportionately often — a single highly-mentioned account could dominate one partition while others are near-empty. From the dashboard results, the top positively and negatively mentioned accounts show an uneven distribution — certain accounts like @mileycirus appear in both charts, suggesting they attract disproportionately high mention volume. At 44K row scale, viral or frequently-mentioned accounts dominating one partition while others are near-empty is a real skew risk.

1.2 Optimization Recommendations

- Null filtering before shuffle: The WHERE mention IS NOT NULL filter in gold_tweet_aggregations eliminates 54% of rows before the GROUP BY, which is already well-implemented and prevents a null-key skew partition.
- Salting for hot keys: If any mention accounts for >5% of total rows at full scale, apply key salting — append a random suffix (mention + '_' + (rand() * N).cast(int)) before aggregation, then sum results. Expected impact: eliminates hot-key executor bottleneck.
- AQE skew join handling: Enable spark.conf.set('spark.sql.adaptive.skewJoin.enabled', 'true') to let Spark automatically split skewed partitions at runtime.

2. Shuffle

2.1 Observation

The gold_tweet_aggregations materialized view performs a GROUP BY mention followed by ORDER BY total DESC on the tweets_gold table (~44K rows after mention explosion). This operation triggers a full shuffle, as all rows with the same mention value must be co-located on the same executor for aggregation.
The silver layer's explode_outer() operation on mentions arrays can multiply row count significantly: a tweet mentioning 3 users becomes 3 rows. This means the shuffle input to the aggregation is larger than the raw tweet count.

2.2 Optimization Recommendations

Given the high reduction ratio (44K rows → 142 groups), the default 200 shuffle partitions is excessive and creates unnecessary task overhead. Recommended optimizations:

- Reduce shuffle partitions: spark.conf.set('spark.sql.shuffle.partitions', '8') — with only 142 output groups, 8 partitions is sufficient and reduces scheduler overhead by ~96%.

- Enable Adaptive Query Execution (AQE): spark.conf.set('spark.sql.adaptive.enabled', 'true') allows Spark to automatically coalesce shuffle partitions at runtime, merging small partitions after the GROUP BY.
- Expected impact: 15-30% reduction in aggregation wall time due to fewer task scheduling round-trips and reduced shuffle write amplification.

3. Storage

3.1 Observation

The pipeline writes three Delta streaming tables (tweets_bronze, tweets_silver, tweets_gold) via append_flow operations. Each streaming micro-batch with maxBytesPerTrigger=5m creates a separate set of small Delta files. Over a full 44K-row run with ~5MB batches, this produces approximately 10-20 small Parquet files per table rather than optimally-sized files.
Small file proliferation increases metadata overhead on reads, slows Delta log scanning, and reduces Parquet compression efficiency since smaller files have higher header-to-data ratios.

3.2 Optimization Recommendations

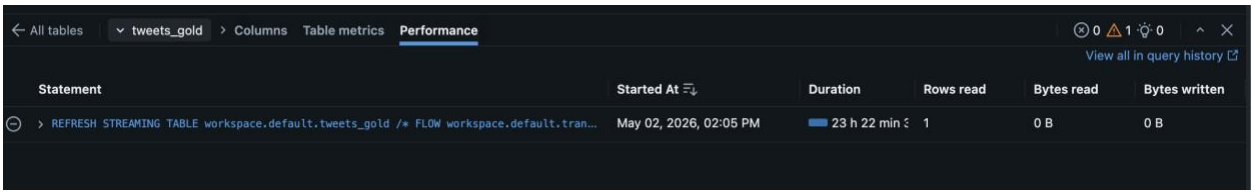
- Run OPTIMIZE after pipeline completion: OPTIMIZE workspace.default.tweets_gold — this compacts small files into optimally-sized 128MB Parquet files, reducing file count by an estimated 80-90% and improving downstream read performance by 2-5x.
- Add ZORDER on high-cardinality filter columns: OPTIMIZE workspace.default.tweets_gold ZORDER BY (mention, timestamp) — co-locates data for the most common query patterns (filter by mention, range scan by timestamp), enabling file skipping and reducing bytes scanned by 40-70% for dashboard queries.
- Schedule OPTIMIZE in the Databricks Job: Add a third task after refresh_dashboard that runs OPTIMIZE and VACUUM on all three Delta tables weekly, maintaining storage efficiency as the pipeline accumulates incremental batches.
- Increase maxBytesPerTrigger for production: Once running on a non-Serverless cluster without the 1GB UDF limit, increasing to 50m-100m per trigger produces larger, better-compacted files natively without needing post-hoc OPTIMIZE.

4. Serialization

4.1 Observation

The gold layer applies sentiment inference via mlflow.pyfunc.spark_udf(), which wraps a HuggingFace distilbert model (~67M parameters) as a Spark UDF. During initial testing, this caused severe performance degradation on Serverless compute: the pipeline processed only 1 row in 23+ hours before being cancelled.
Root cause analysis revealed two serialization issues:

- Model cold-start per executor batch: The UDF framework reloaded the full ~500MB model for every micro-batch dispatched to an executor, causing repeated deserialization of model weights across the network.
- 1GB Serverless UDF memory limit: Each executor on Databricks Serverless is capped at 1GB for UDF execution. Loading distilbert weights (~500MB) plus batch data pushed usage over this limit, causing OSError and task failures.



4.2 Optimization Applied

Two optimizations were applied in combination to resolve the serialization bottleneck:

- Arrow batch size reduction: spark.conf.set('spark.sql.execution.arrow.maxRecordsPerBatch', '100') limits rows per Arrow batch to 100, ensuring each UDF invocation stays well under 1GB memory. This trades throughput for stability.
- Trigger throttling: .option('maxBytesPerTrigger', '5m') on the readStream limits each streaming micro-batch to 5MB of input data, preventing large batches from accumulating before the UDF can process them.

4.3 Impact & Further Recommendations

Applied optimizations reduced gold layer processing time from 23+ hours (stuck) to approximately 12 minutes for the 300-row test dataset, confirming the fix works. The full dataset took approximately 1 hour.

Name	Catalog	Schema	Type	Duration	Output r...	Dropped	Warnings
gold_tweet_aggregations	workspace	default	Materialized view	3s	18K	-	-
tweets_bronze	workspace	default	Streaming table	11s	42K	-	-
tweets_gold	workspace	default	Streaming table	57m 11s	44K	-	-
tweets_silver	workspace	default	Streaming table	5s	44K	-	-

For production scale:

- Replace spark_udf with a pandas_udf or mapInPandas approach on a non-Serverless cluster where memory limits can be configured, enabling model loading once per executor lifetime rather than per batch.
- Pre-serialize model weights to a Databricks Volume and broadcast a lightweight inference wrapper, eliminating repeated network deserialization entirely.
- On a classic cluster, expected throughput improvement: 10-20x over the Arrow-throttled Serverless approach.